

Cours MMI 3

Introduction

Licence professionnelle MMI 3 · Jérémy Chauffourier

Jérémy Chauffourier

Développeur fullstack chez [Opsone](#)

 Opsone, agence digitale à Champs-sur-Marne

 +20 ans de plateformes sur-mesure

 Node.js · Ruby · PHP

 github.com/batosai

 jeremy.chauffourier@opsone.net

Conseil & audit → UX/UI → développement front & back →
DevOps → exploitation long terme

Ce que vous avez déjà vu

HTML / CSS

intégration, responsive

JavaScript

DOM, événements, fetch

PHP

avec ou sans framework

CMS

WordPress & co.

Bases de données

SQL, modélisation

Git

solo ou en équipe

UX / UI

maquettage, Figma

Framework front

Vue, React...

Mise en ligne

hébergement, FTP, serveur

Dev mobile

natif, React Native, PWA

Autres ?

ce que j'ai oublié

Vos niveaux

	Jamais touché	Déjà bricolé	À l'aise
JavaScript (ES moderne)	☐	☐	☐
TypeScript	☐	☐	☐
Node.js	☐	☐	☐
npm / gestion de paquets	☐	☐	☐
Un framework front (Vue, React...)	☐	☐	☐
Un framework back (Express, Nest, AdonisJS...)	☐	☐	☐
Base de données / SQL	☐	☐	☐
Git en équipe	☐	☐	☐

Ce qu'on va clarifier ensemble

Des notions floues pour beaucoup de monde

- Un **CMS** (WordPress...) et un **framework** : deux outils, deux métiers
- Un **framework**, à quoi ça sert vraiment ? Ce qu'il fait pour vous, ce qu'il vous impose
- **JavaScript** et **TypeScript** : pourquoi typer ?
- **Node.js** : un moteur, un serveur, ou juste `npm install` ?
- **Front** et **back** : qui fait quoi, et où s'arrête la frontière

Frameworks & bibliothèques, vue d'ensemble

Chaque langage a les siens

Back	Front	Moteurs de template	Mobile	Gestion de paquets
PHP · Symfony, Laravel	React · le plus répandu	Twig · Symfony	React Native · JS	npm, pnpm, yarn · Node
Ruby · Ruby on Rails	Vue · doux à apprendre	Blade · Laravel	Flutter · Dart	Composer · PHP
Python · Django, FastAPI	Angular · très cadré, entreprise	ERB · Rails	Swift / Kotlin · natif	Bundler · Ruby
Java · Spring	Svelte · compilé, minimaliste	Jinja · Django		pip, uv · Python
C# · ASP.NET		Edge · AdonisJS		Maven · Java
		EJS, Pug · Node		Cargo · Rust

Partout les mêmes briques : **routing, accès aux données, authentification, templates, tests**. Apprendre un framework sérieusement, c'est apprendre à lire tous les autres.

Les frameworks Node.js

Même langage côté serveur et côté navigateur, c'est là toute la bascule

Back - minimalistes

Express · le standard de fait, tout est à assembler

Fastify · même esprit, plus rapide et plus moderne

Hono · ultra léger, pensé pour l'edge

Liberté totale, donc structure à votre charge.

Full-stack (front + serveur)

Next.js (React) · **Nuxt** (Vue) · **SvelteKit** (Svelte) · **Astro**
(contenu)

Back - complets

NestJS · architecture imposée, très typée (esprit
Angular/Spring)

AdonisJS · tout inclus : ORM, auth, validation (esprit
Laravel/Rails)

Décisions déjà prises, on gagne du temps à l'échelle.

L'outillage autour

Vite · build & dev server

Prisma / Lucid / TypeORM · accès base de données

Vitest · tests

Ce que nous allons voir

Le développement serveur avec Node.js

Node.js

Le JavaScript qui s'exécute **hors du navigateur**.
Votre code tourne sur un serveur : il reçoit des requêtes, parle à une base de données, renvoie des réponses.

AdonisJS

Un framework **complet** : routage, base de données, authentification, validation, templates, tests.
Tout est fourni et cohérent : on code le métier, pas la plomberie.

Comprendre

ce qui se passe entre le navigateur
et le serveur

Construire

une application complète, de la
route à la base de données

Mettre en ligne

et tenir le code dans la durée

AdonisJS

Un framework back Node.js complet, en TypeScript, l'esprit Laravel / Rails

Le cœur

Routing & contrôleurs · une URL → une méthode

Lucid · l'ORM, vos tables vues comme des objets

VineJS · validation des données entrantes

Auth & Bouncer · connexion et droits d'accès

Autour

Edge · moteur de templates

Ace · la ligne de commande du projet

Japa · tests intégrés

Mailer, queues, événements · fournis, pas à chercher

La philosophie : tout est fourni, cohérent et documenté au même endroit. Pas de chasse aux bibliothèques, pas de débat d'arborescence, les conventions sont posées, et **TypeScript** est natif de bout en bout : votre éditeur connaît vos routes, vos modèles et vos règles de validation.

Projet open source mature, créé par Harminder Virk · communauté francophone très active · adonisjs.com

AdonisJS à géométrie variable

Un même framework, plusieurs formes de projet - le choix se fait au démarrage

API seule

Pas de front du tout : le serveur ne renvoie que du **JSON**.
Pour alimenter une application mobile, un front séparé, ou d'autres services.

Web classique : Edge

Le serveur génère le **HTML** et le renvoie tout prêt.
Simple, rapide, excellent pour le SEO. Le socle historique du web.

Inertia

Un front **Vue** ou **React** branché directement sur vos contrôleurs.
L'interface moderne sans écrire ni consommer d'API.

Monorepo

API et front **séparés mais dans le même dépôt**.
Chacun son cycle de vie, les types partagés d'un bout à l'autre.

Et l'architecture reste la vôtre. Adonis propose des conventions, il ne les impose pas jusqu'au bout : arborescence configurable, découpage en modules métier, services et providers maison. On part du cadre fourni, on l'adapte quand le projet le réclame, pas avant.

L'architecture de base

Une place pour chaque chose, les conventions qui rendent un projet lisible par tous

app/ controllers/ models/ middleware/ validators/ config/ database/ migrations/ seeders/ start/ routes.ts resources/ public/ tests/ bin/ adonisrc.ts	app/ : votre métier. Contrôleurs (ce qu'on fait à chaque URL), modèles (vos tables vues comme des objets), middleware (ce qui s'exécute avant), validateurs (ce qu'on accepte en entrée). config/ : un fichier par sujet : base de données, session, authentification, mail. Jamais de secret ici, ils vivent dans <code>.env</code> . database/ : l'histoire de votre base. Les migrations versionnent chaque changement de schéma, les seeders remplissent des données de départ. start/ : ce qui se charge au démarrage. <code>routes.ts</code> est la table des matières de l'application : toutes les URL y sont déclarées. resources/ : les templates Edge et les assets à compiler. (Avec Inertia, le front vit dans <code>inertia/</code> .) public/ : servi tel quel au navigateur : favicon, images, PDF. tests/ : <code>unit/</code> et <code>functional/</code> . bin/ : les points d'entrée (serveur, console, tests).
---	--

Ace, la console du projet

Un seul outil en ligne de commande, livré avec l'application, et qui la connaît

Démarrer et explorer

```
node ace serve --watch    # lance le serveur de dev
node ace list              # toutes les commandes disponibles
node ace repl              # une console dans votre application
```

Générer du code

```
node ace make:controller Movie
node ace make:model Review --migration
node ace make:middleware Auth
node ace make:validator Movie
```

Installer un module officiel

```
node ace add @adonisjs/auth
```

Source : docs.adonisjs.com/guides/ace

Ce qui le distingue d'un script npm, Ace tourne *dans* votre application : il a accès à la configuration, à la base de données, aux modèles. D'où le REPL, où l'on manipule ses vraies données sans écrire de contrôleur.

`make:` ne fait pas de magie : il écrit un fichier vide au bon endroit, avec le bon nom et les bons imports. Le gain, c'est la convention respectée sans y penser.

`add` fait trois choses d'un coup : installer le paquet, publier sa configuration, enregistrer le provider. Le même geste pour auth, lucid, i18n ou le limiter.

Et vos propres commandes. `node ace make:command import_movies` et vous ajoutez votre verbe à la liste : import, nettoyage, tâche planifiée. On y reviendra en séance queues.

Travaux dirigés

Installer, explorer, modifier

TD : Installer les quatre kits

Installez les quatre kits, comparez-les vous-même

Prérequis

```
node -v # Node 24 minimum
```

Les quatre projets

```
npm create adonisjs@latest app-hypermedia -- --kit=hypermedia
npm create adonisjs@latest app-vue -- --kit=vue
npm create adonisjs@latest app-react -- --kit=react
npm create adonisjs@latest app-api -- --kit=api
```

Puis, un seul à la fois

```
cd app-hypermedia && npm run dev
```

Sur `http://localhost:3333` → créez-vous un compte.

Ce que chacun apporte

hypermedia : Edge + Alpine.js

vue : Inertia + Vue

react : Inertia + React

api : monorepo, JSON seul

Tous arrivent avec **SQLite** et l'**authentification** déjà configurées : inscription et connexion fonctionnent avant d'écrire une ligne de code.

Les quatre écoutent sur le **port 3333**. Lancez-en un seul, ou changez `PORT` dans le `.env` des autres.

TD : Comparer

Quatre dossiers ouverts côte à côte : le même framework, quatre visages

Ce qui est identique partout

`app/` , `config/` , `database/` , `start/routes.ts` : le back ne change pas.

Ce qui change

`resources/views/` chez Hypermedia · `inertia/pages/` chez Vue et React · `apps/backend` + `apps/frontend` pour l'API.

À observer ensemble : ouvrez `start/routes.ts` dans vos quatre projets. Les routes sont les mêmes. Ce qui diffère, c'est uniquement **ce que le contrôleur renvoie** : du HTML, une page Inertia, ou du JSON.

TD : Modifier le modèle User

Le kit fournit un champ `full_name` . On veut un prénom, un nom, et un rôle.

1. Créer la migration

```
node ace make:migration add-last-name-and-first-name-to-users --alter=users
```

2. Décrire le changement : dans `database/migrations/xxxx_alter_users_table.ts` , une classe qui `extends BaseSchema` avec `protected tableName = 'users'` .

```
// Aller : ce qu'on met en place
async up() {
  this.schema.alterTable(this.tableName, (table) => {
    table.dropColumn('full_name')
    table.string('first_name').nullable()
    table.string('last_name').nullable()
    table.enum('role', ['user', 'admin'])
      .nullable().defaultTo('user')
    table.timestamp('last_login_at')
  })
}
```

```
// Retour : exactement l'inverse
async down() {
  this.schema.alterTable(this.tableName, (table) => {
    table.string('full_name').nullable()
    table.dropColumn('first_name')
    table.dropColumn('last_name')
    table.dropColumn('role')
    table.dropColumn('last_login_at')
  })
}
```

TD : Appliquer et vérifier

3. Jouer la migration

```
node ace migration:run
node ace migration:status
```

4. Regarder ce qui a changé

```
// database/schema.ts : régénéré tout seul
declare firstName: string
declare lastName: string
declare role: string
```

Vous n'écrivez jamais dans `schema.ts` : il est reconstruit à chaque migration. Votre code va dans `app/models/user.ts` .

5. En cas d'erreur

```
migration:rollback : annule la dernière migration
migration:fresh : repart d'une base vide
node ace list : toutes les commandes disponibles
```

Et pour voir la base

```
node ace repl
> const { default: User } = await import('#models/user')
> await User.all()
```

TD : Le nom complet et le rôle

Une propriété calculée ne vit pas en base : elle se déduit des colonnes existantes.

```
// app/models/user.ts
import { UserSchema } from '#database/schema'
import hash from '@adonisjs/core/services/hash'
import { compose } from '@adonisjs/core/helpers'
import { withAuthFinder } from '@adonisjs/auth/mixins/lucid'

export const Role = {
  USER: 'user',
  ADMIN: 'admin',
} as const

export type Role = (typeof Role)[keyof typeof Role]

export default class User extends compose(UserSchema, withAuthFinder(hash)) {
  get fullName(): string {
    return `${this.firstName} ${this.lastName}`.trim()
  }

  get isAdmin(): boolean {
    return this.role === Role.ADMIN
  }
}
```

TD : Adapter le formulaire d'inscription

La colonne a changé : tout ce qui la remplissait doit suivre

Le validateur

```
// app/validators/user.ts
export const signupValidator = vine.create({
  firstName: vine.string().trim().maxLength(64)
    .nullable().optional(),
  lastName: vine.string().trim().maxLength(64)
    .nullable().optional(),

  // email, password et la confirmation
  // restent inchangés
})
```

Le champ `fullName` fourni par le kit disparaît, remplacé par les deux colonnes créées à l'étape précédente.

1. Le formulaire : deux champs à la place d'un. Selon le kit que vous poursuivez, il est dans

`inertia/pages/auth/signup.tsx` ou dans `resources/views/`.

2. Le contrôleur : rien à changer.

`NewAccountController.store` fait `User.create({ ...payload })` : c'est le validateur qui décide de ce qui entre.

3. Vérifier : inscrivez-vous, puis affichez `fullName` dans la barre de navigation. La propriété calculée sert enfin à quelque chose.